
Data Structures

BS (CS) _Fall_2025

Lab_10 Manual



Learning Objectives:

1. Binary Tree
2. Binary Tree to Array and Array to binary tree conversion

Binary Tree

A Binary Tree is a hierarchical data structure in which each node can have at most two children: a left child and a right child.

Node Structure

A Binary Tree can be implemented using a linked structure, where each node contains:

- data : stores the value or information
- left : pointer/reference to the left child
- right : pointer/reference to the right child

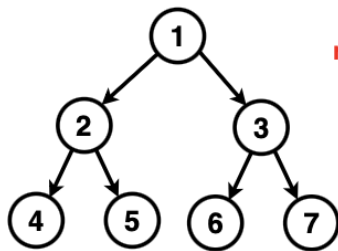
```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
  
    Node(int val) {  
        data = val;  
        left = right = nullptr;  
    }  
};
```

Binary Tree Properties

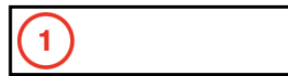
- The topmost node is called the root.
- Each node may have 0, 1, or 2 children.
- Nodes with no children are called leaf nodes.
- The height of a tree is the number of edges on the longest path from the root to a leaf.

Types of Binary Trees

- **Full Binary Tree:** Every node has either 0 or 2 children.
- **Complete Binary Tree:** All levels are completely filled except possibly the last, which is filled from left to right.
- **Perfect Binary Tree:** All internal nodes have 2 children and all leaves are at the same level.
- **Skewed Binary Tree:** All nodes are aligned to one side (left or right).



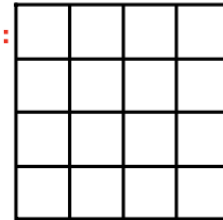
Enqueue the root node in the queue



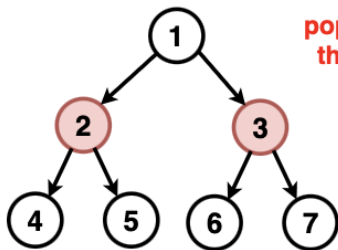
queue <TreeNode*>

size = 1

level 1:



level order traversal vector



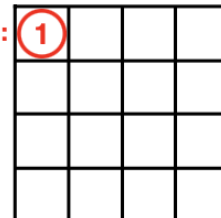
pop nodes of level 1 and add their left and right children



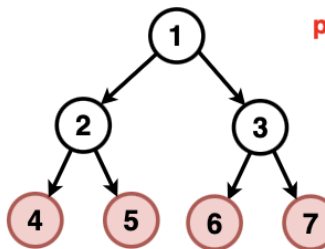
queue <TreeNode*>

size = 2

level 1:



level order traversal vector



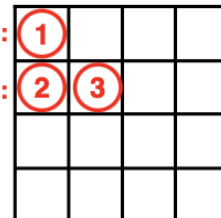
pop nodes of level 2 and add their left and right children



queue <TreeNode*>

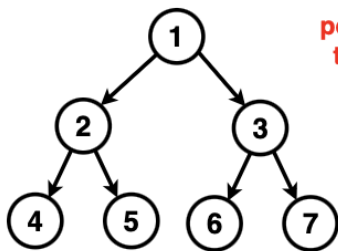
size = 4

level 1:

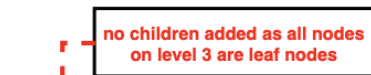


level 2:

level order traversal vector



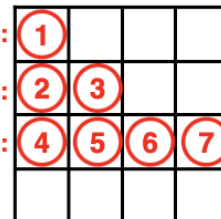
pop nodes of level 3 and add their left and right children



queue <TreeNode*>

size = 0

level 1:



level 2:

level 3:

level order traversal vector

Basic Binary Tree Operations

1. Insertion (Level-Order)

In a Level-Order Binary Tree, new nodes are inserted in a breadth-first manner — filling the tree level by level, from left to right.

This ensures the tree always remains complete, meaning no gaps appear between nodes at any level.

Steps:

1. If the tree is empty, create a new node as the root.
2. Otherwise, start from the root and traverse the tree level by level using a queue.
3. For each node:
 - If the left child is empty → insert the new node there.
 - Else if the right child is empty → insert the new node there.
 - Otherwise, move both children into the queue and continue checking the next node.
4. The process stops as soon as the first available position (left-to-right) is found.

2. Level-Order Traversal (Breadth-First Traversal)

Concept:

Unlike the depth-first traversals (Inorder, Preorder, Postorder), the Level-Order Traversal visits nodes level by level, starting from the root and moving left to right on each level.

It uses a queue to keep track of nodes ensuring all nodes on one level are visited before moving to the next.

Steps:

1. Create an empty queue and enqueue the root node.
2. While the queue is not empty:
 - Dequeue the front node and print its value.
 - Enqueue the left child (if it exists).
 - Enqueue the right child (if it exists).
3. Continue until all nodes have been visited.

Code:

```
void levelOrder(Node* root) {  
    if (root == nullptr) return;  
    queue<Node*> q;  
    q.push(root);  
  
    while (!q.empty()) {
```

```

Node* current = q.front();
q.pop();

cout << current->data << " ";

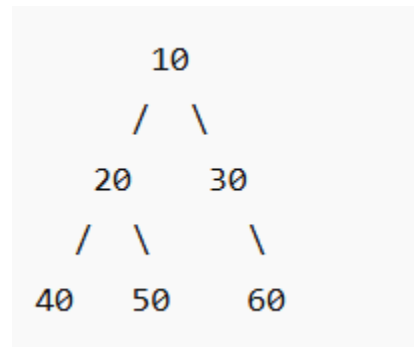
if (current->left != nullptr)
    q.push(current->left);

if (current->right != nullptr)
    q.push(current->right);
}
}

```

Example Output:

For this tree:



Level-Order Traversal Output:

[10 20 30 40 50 60]

3. Searching (Level-Order Binary Tree)

Concept:

In a Level-Order Binary Tree, searching is done using a breadth-first approach, where nodes are checked level by level from top to bottom and left to right.

Steps:

1. If the tree is empty, return false.
2. Create a queue and enqueue the root node.
3. While the queue is not empty:
 - Dequeue the front node.
 - If the current node's data matches the target value, return true.
 - Enqueue the left child (if it exists).

- Enqueue the right child (if it exists).
4. If all nodes are visited and the value is not found, return false.

4. Deletion (Level-Order Binary Tree)

Concept:

In a Level-Order Binary Tree, deletion is performed in such a way that the complete structure of the tree (filled from left to right) is preserved.

To do this, we replace the target node's data with the deepest (last) node's data, and then delete that deepest node.

This ensures there are no gaps in the level-order structure after deletion.

Steps:

DeleteNode(root, key)

1. **Check for empty tree:**
 - If root is NULL, display "Tree is empty" and stop.
2. **Find the target node (the one to delete):**
 - Perform a level-order traversal using a queue.
 - Keep track of both the target node (the one matching the key) and the last (deepest) node visited.
 - If the target value is not found, display "Value not found" and stop.
3. **Replace data:**
 - Copy the data of the deepest node into the target node (the one to delete).
4. **Delete the deepest node:**
 - Perform another level-order traversal to locate the parent of the deepest node.
 - Once found, set that child pointer (left or right) to NULL.
5. **Display confirmation message:**
 - Print "Node deleted successfully" once the operation completes

Time Complexities

Operation	Best Case	Average Case	Worst Case	Description
Insertion	$O(1)$	$O(n)$	$O(n)$	Traverse level-by-level until the first empty position is found and insert there.
Search	$O(1)$	$O(n)$	$O(n)$	Perform a level-order search; may need to check every node in the tree.
Traversal	$O(n)$	$O(n)$	$O(n)$	Visit all nodes exactly once in level-order sequence.
Deletion	$O(1)$	$O(n)$	$O(n)$	Find target and deepest node using level-order traversal, replace data, then delete deepest node.

Conversion between Array and Binary Tree

Concept

A binary tree can be built directly from an array using index relationships.

Each element of the array represents a node in the tree, where the position in the array defines its parent-child relationships.

The tree is filled level by level, left to right, making it a complete binary tree.

Index Relationships

For an element at index i in the array:

- Left Child Index = $2 * i + 1$
- Right Child Index = $2 * i + 2$
- Parent Index = $(i - 1) / 2$ (integer division)

Steps

1. Start with an integer array (e.g., [10, 20, 30, 40, 50, 60]).
2. Create the root node using the element at index 0.
3. For each element i :
 - Use the index formulas to connect the node to its left and right children.
4. Repeat until all elements are linked.
5. The resulting tree will be complete and filled in level order.

Example

Array:

[10, 20, 30, 40, 50, 60]

Tree:

